Guía de estudio y defensa

Evacuación de los bloques G y H (ESPE, campus Matriz) con Optimización por Colonia de Hormigas (ACO)

Aplicaciones basadas en el conocimiento · NRC 36859 · Maycol Celi, Erika Guayanay, Alejandro Obando

Documento interno de estudio: no es para entregar. Explica todo desde cero para que los tres puedan exponer y defender el trabajo.

> **Cómo usar esta guía.** Parte 1: qué dice cada quien en la presentación (guion de 2 minutos por persona). Partes 2 y 3: conceptos básicos y modelo del problema. Parte 4: Dijkstra. Partes 5 y 6: ACO, ACO clásico y ACO guiado. Parte 7: el agente. Parte 8: cómo funciona el programa de Python y cómo probarlo en clase. Parte 9: qué medimos. Parte 10: cada gráfica explicada. Parte 11: qué significan los resultados. Parte 12: límites. Parte 13: preguntas probables con respuesta. Parte 14: glosario.

0. El proyecto en un minuto

Después de un sismo, las personas que están en los bloques G y H del campus deben llegar a un punto de encuentro. Modelamos los bloques como un **grafo** (75 nodos y 79 aristas) y buscamos la mejor ruta desde cada una de las **42 aulas u oficinas de origen**. Comparamos tres métodos:

Método	Idea en una línea	Resultado
Dijkstra	Algoritmo exacto: siempre expande el nodo de menor costo acumulado.	Siempre óptimo, muy rápido (0,08 a 0,09 ms).
ACO clásico	Hormigas virtuales que caminan al azar y dejan feromona; sin trucos.	Solo 11 a 22 % de las corridas encuentra ruta; cuando la encuentra, suele ser peor que la óptima.
ACO guiado	Lo mismo, pero con poda de callejones, retroceso y una heurística tipo A*.	100 % de éxito y 0 % de diferencia con el óptimo, pero ~264 veces más lento que Dijkstra.

Además hay un **agente de evacuación** (percibe bloqueos, decide la ruta con ACO, actúa dando instrucciones) y una **app en Streamlit** para simular en vivo.

> **La idea que hay que defender con honestidad:** ACO guiado llega al óptimo, pero no gracias a las feromonas, sino a la poda, el retroceso y la heurística. Dijkstra es mejor para este problema (pequeño y fijo). Usamos ACO para estudiar cómo se comporta una metaheurística, no porque sea necesaria aquí.

1. Reparto de la presentación y guion (12 láminas, 4 por persona)

La presentación del PDF tiene 12 láminas. Cada persona tiene 2 minutos, es decir unos 30 segundos por lámina (~250 a 280 palabras en total). Abajo hay un guion sugerido, lo que hay que saber de cada lámina y qué le pueden preguntar. Pueden decidir entre ustedes quién es el expositor 2 y el 3.

Expositor 1 (Maycol): láminas 1 a 4 · contexto, problema y modelo

Lámina 1, Portada. «Buenos días. Somos el grupo que modeló la evacuación de los bloques G y H del campus Matriz. Usamos un grafo de 75 nodos y 42 orígenes, y comparamos tres métodos: Dijkstra, ACO clásico y ACO guiado, en seis escenarios.»

Lámina 2, Problema y objetivo. «Tras un sismo, 42 orígenes, que son aulas y oficinas, deben llegar a uno de tres puntos de encuentro. Los puntos de encuentro son hipotéticos, porque no hay un plan oficial publicado para estos bloques. Una grada o una salida puede bloquearse y obligar a cambiar de ruta. El objetivo es medir cuatro cosas: éxito, costo frente al óptimo, convergencia y tiempo de cálculo, y además cuánto tarda en llegar la población, 1020 personas, en seis escenarios.»

Lámina 3, Grafo y supuestos. «Calibramos el plano con la distancia entre las dos salidas, 61,7 metros: da 6,948 píxeles por metro y un error de 2,19 %. El grafo tiene 75 nodos y 79 aristas: 42 orígenes, 15 pasillos y 10 nodos de gradas. El costo de una arista es su distancia; las gradas cuestan el doble y los tramos techados una vez y media. Supusimos 25 personas por aula, 60 segundos de reacción y 1 segundo entre una persona y la siguiente.»

Lámina 4, Rutas sobre el plano. «Aquí vemos tres orígenes, desde planta baja, piso 2 y piso 4. Dijkstra y ACO guiado coinciden: bajan por las gradas norte, salen por la Salida B y llegan al punto PE-2. En cambio, desde H203 el ACO clásico se desvía a la Salida A y termina en PE-1.»

Lámina	Lo que debes saber	Pregunta probable
2	42 orígenes = 40 aulas (G y H, pisos 1 a 4, 5 aulas por piso por bloque) + 2 nodos de planta baja (oficinas de H y laboratorio de G). Población = 40×25 + 2×10 = **1020**.	¿Por qué los puntos de encuentro son hipotéticos? Porque no tenemos el plan oficial; los pusimos en vías exteriores para poder terminar la ruta. Es un límite declarado.
3	Calibración: usamos las dos marcas «X» de las salidas (61,7 m reales entre ellas) para obtener 6,948 px/m, y la verificamos con la barra de escala del plano (71 px = 10 m): error 2,19 %. Las gradas costarán ×2 por esfuerzo, el techado ×1,5 por modelo de penalización. El dibujo de nodos de la lámina es solo ilustrativo.	¿Por qué gradas ×2? Es un supuesto de modelado (subir o bajar cuesta más que caminar plano); está en `config_modelo.json` y se puede cambiar.
4	Los tres paneles son la misma figura (fig. 1). El segmento largo de H203 es el camino del ACO clásico fuera del recorte del plano hacia PE-1. Dijkstra y guiado dan siempre la misma ruta.	¿Por qué todos van por la Salida B? Porque el PE-2 queda cerca de B y no modelamos colas; ver parte 12.

Expositor 2: láminas 5 a 8 · agente, métodos y resultados principales

Lámina 5, Agente de evacuación. «Es un agente basado en metas y en utilidad. Percibe las aristas bloqueadas y la congestión, decide la ruta con ACO, actúa dando instrucciones y tiempo estimado, y avisa si no hay ruta. El entorno es parcialmente observable y dinámico. Hemos delimitado el agente: no simula interacción entre las 1020 personas y no aprende entre simulacros.»

Lámina 6, Tres métodos, tres lógicas. «Dijkstra es la referencia exacta: expande siempre el menor costo acumulado y tarda unos 0,08 milisegundos. ACO clásico usa 20 hormigas por 100 iteraciones, sin poda ni retroceso, con visibilidad uno sobre costo, alfa 1, beta 2, rho 0,5 y Q igual a 1. ACO guiado usa las mismas 20 por 100, pero poda las aulas, retrocede cuando se atasca y agrega a la visibilidad la distancia a la meta, con beta 1. No llama a Dijkstra.»

Lámina 7, Éxito y calidad de rutas. «Probamos seis escenarios por 42 orígenes por 30 corridas: 15.120 pruebas. ACO guiado logra 100 % de éxito, 7560 de 7560, y 0 % de brecha frente a Dijkstra. ACO clásico solo logra entre 11 y 22 % de éxito, con una brecha media de 5 a 17 % calculada solo sobre las corridas que sí llegaron; el peor caso llega a 157 %.»

Lámina 8, Convergencia. «El guiado llega al óptimo en 1 a 4 iteraciones. El clásico se atasca en callejones: desde H203 halla ruta en 8 de 30 corridas, y desde H403 en 5 de 30. La poda y el retroceso explican el desempeño del guiado.»

Lámina	Lo que debes saber	Pregunta probable
5	PEAS: Rendimiento (llegar a un PE, minimizar costo, avisar si no hay ruta), Entorno (grafo de G y H), Actuadores (`decidir()` y `actuar()`), Sensores (`percibir()`: bloqueos y congestión). Detalle en la parte 7. Parcialmente observable: solo sabe lo que se le informa. Dinámico: los bloqueos cambian. Basado en metas (llegar a un PE) y utilidad (minimizar costo).	¿Es un agente simple de reflejo? No: planifica una ruta completa. ¿Aprende? No, solo reutiliza feromona (warm start) dentro de la misma sesión.
6	Las partes 4, 5 y 6 de esta guía explican Dijkstra, ACO y sus variantes. Memoriza los parámetros: 20 hormigas, 100 iteraciones, α=1, β=2 (clásico) y β=1 (guiado), ρ=0,5, Q=1.	¿Por qué β=2 en el clásico? Es el valor estándar de la literatura (Dorigo y Stützle) y lo fijamos antes de medir, para no «afinarlo» a favor de nadie.
7	El 100 % es sobre 7560 corridas (6 × 42 × 30). La brecha del clásico se mide solo en las exitosas, por eso es optimista: la tasa de éxito de 11 a 22 % es el dato más importante.	¿Es justa la comparación? Sí en tiempo y número de hormigas, pero el guiado tiene más conocimiento del problema (poda, retroceso, heurística).
8	Eje X: iteración; eje Y: mejor costo acumulado; línea verde punteada: óptimo de Dijkstra (161,0, 192,2 y 223,4). Banda: ±1 desviación estándar. «Corridas con ruta» indica cuántas de 30 hallaron camino.	¿Por qué el clásico empeora (sube) en H203? Cuando una corrida nueva encuentra su primera ruta, suele ser larga y la media sube al sumarse.

Expositor 3: láminas 9 a 12 · sensibilidad, adaptación, evacuación y conclusiones

Lámina 9, Sensibilidad de parámetros. «Con 10 orígenes, el parámetro beta domina: con beta 0 el éxito es de 99 %, con beta 2 baja a 34 % y con beta 5 a 14 %. Con Q igual a 1, alfa y rho no cambian el 34 %. Con Q igual a 1000 la brecha sube a 16,34 %.»

Lámina 10, Bloqueos y warm start. «Si se bloquea la Salida B, o las gradas norte, todo el flujo va a la Salida A: el costo sube 23 % y la llegada de la última persona 39 segundos. Reutilizar la feromona anterior, el warm start, ahorra cerca de 20 % del tiempo en ACO guiado cuando la ruta principal está libre. Pero con la Salida B bloqueada, la convergencia empeora de 1,67 a 5,04 iteraciones, porque el rastro previo apunta a la ruta cerrada.»

Lámina 11, Llegadas de 1020 personas. «En el escenario normal la primera persona llega a los 194 segundos y la última a los 272. Con B cerrada, a los 233 y 311. Dijkstra y ACO guiado dan la misma curva; el clásico completa solo 770 o 895 llegadas cuando hay bloqueos del norte.»

Lámina 12, Conclusiones. «Dijkstra es unas 264 veces más rápido: 0,08 contra 13 a 30 milisegundos. ACO guiado alcanza el óptimo en todas las pruebas, y se lo debemos más a la poda y al retroceso que a las feromonas. Es un modelo experimental, no un plan oficial. Faltan colas, capacidad de gradas y validación en campo con simulacros.»

Lámina	Lo que debes saber	Pregunta probable
9	La sensibilidad es solo del **ACO clásico**, escenario Normal, 10 orígenes, 30 corridas por valor. β=0: 99,3 % éxito; β=1: 76,3 %; β=2: 34,3 %; β=3: 21 %; β=5: 14,3 %. Más hormigas: 5→22 %, 50→51 % (pero cuesta más tiempo: 2,3 ms→17 ms). α y ρ: 34,3 % siempre.	¿Por qué β=0 funciona mejor? Sin visibilidad, la hormiga elige casi al azar entre vecinos y no se deja atraer por las aristas cortas de las aulas (callejones). Ver partes 5 y 6.
10	Warm start = empezar con la feromona de la corrida anterior en vez de uniforme. Reducción de tiempo medida: 23,9 % (Salida A bloqueada) y 20,3 % (congestión). Con B bloqueada empeora: 1,67→5,04 iteraciones.	¿Entonces conviene el warm start? Depende: ayuda si la ruta principal sigue libre; estorba si el rastro apunta a una ruta ya cerrada.
11	Primera persona = 60 s de reacción + tiempo de la ruta (costo ÷ 1,2 m/s). Ejemplo: costo 161 ÷ 1,2 = 134 s; 134 + 60 = 194 s. Última = la ruta más larga (costo 225,8 → 188 s) + 60 + 24 s (la persona 25 del aula). No hay colas.	¿Por qué 194 s y no menos? Porque incluye 60 s de reacción antes de moverse.
12	264× = promedio de los tiempos del guiado en los 6 escenarios (≈21,8 ms) dividido por el de Dijkstra (≈0,083 ms). Los límites ya están declarados.	¿Por qué usar ACO entonces? Para estudiar la metaheurística y como base para problemas más grandes o dinámicos donde Dijkstra deja de ser trivial.

2. Conceptos desde cero

2.1 Grafo, nodo, arista, costo, camino

Un **grafo** es un conjunto de **nodos** (puntos) unidos por **aristas** (conexiones). Aquí un nodo es un lugar (un aula, un pasillo, un tramo de gradas, una salida) y una arista es un tramo que se puede caminar. Cada arista tiene un **costo**: la distancia en metros equivalentes (ver 3.2). Un **camino** es una secuencia de nodos conectados; su costo es la suma de los costos de sus aristas. «Mejor ruta» significa el camino de menor costo desde un origen hasta algún punto de encuentro.

2.2 Algoritmo exacto, heurística y metaheurística

Término	Qué es	Ejemplo aquí
Algoritmo exacto	Garantiza la mejor solución.	Dijkstra
Heurística	Regla práctica que estima o guía, sin garantía.	h(j): distancia en línea recta al punto de encuentro más cercano
Metaheurística	Estrategia general de búsqueda inspirada en algo natural; no garantiza el óptimo.	ACO (inspirada en hormigas)
Determinista / estocástico	Siempre da lo mismo / usa azar y puede dar resultados distintos cada vez.	Dijkstra / ACO (por eso hacemos 30 corridas con semillas distintas)

2.3 Semilla y reproducibilidad

ACO usa números aleatorios. La **semilla** fija la secuencia aleatoria: con la misma semilla el resultado se repite exactamente. En los experimentos la corrida i usa la semilla $1000 + i$ ($i = 0$ a 29), igual para las dos variantes, para que la comparación sea justa y repetible.

3. El modelo del problema

3.1 Qué contiene el grafo (75 nodos, 79 aristas)

Tipo de nodo	Cantidad	Qué representa
Orígenes (aulas y planta baja)	42	40 aulas (bloques G y H, pisos 1 a 4, 5 aulas por piso por bloque) y 2 sitios de planta baja (oficinas de H, laboratorio de G)
Pasillos	15	3 por nivel (izquierda, centro, derecha) × 5 niveles (PB, 1, 2, 3, 4)
Gradas	10	2 secciones compartidas (norte y sur) × 5 niveles
Salidas	2	Salida A y Salida B (61,7 m entre ellas)
Vías exteriores	3	Caminos fuera de los bloques
Puntos de encuentro	3	PE-1, PE-2, PE-3, **hipotéticos**

Total: 42 + 15 + 10 + 2 + 3 + 3 = 75. Las aulas son «hojas» del grafo: cada aula está conectada solo a su pasillo. Esto es clave más adelante, porque para una hormiga que no parte de esa aula, entrar a un aula es un **callejón sin salida**.

3.2 Función de costo (la misma para los tres métodos)

```
costo(arista) = distancia_m × (1,5 si la arista es techada, si no 1)
(las gradas ya traen su distancia multiplicada por 2)
tiempo(arista) = costo ÷ velocidad_caminata (1,2 m/s)
```

Por eso el costo se mide en «metros equivalentes»: no son metros reales exactos, sino metros ya penalizados por gradas y techado. La función vive en `src/costos.py` y la usan Dijkstra, ACO y el agente; así la comparación es justa.

3.3 Los seis escenarios

Escenario	Qué se bloquea o altera
Normal	Nada
Gradas norte bloqueadas	Todas las aristas de las gradas norte
Gradas sur bloqueadas	Todas las aristas de las gradas sur
Salida A bloqueada	La arista gradas sur PB → Salida A
Salida B bloqueada	La arista gradas norte PB → Salida B
Congestión en una salida	La arista a la Salida A cuesta ×3 (factor de congestión)

3.4 Supuestos del modelo (todos en `data/config_modelo.json`)

Variable	Valor	Significado
poblacion_por_aula	25	Personas por aula (los 2 sitios de PB tienen 10 cada uno)
velocidad_caminata_m_s	1,2	Velocidad de caminata
tiempo_reaccion_s	60	Segundos antes de empezar a moverse
tiempo_entre_personas_s	1,0	Intervalo entre una persona y la siguiente del mismo origen
factor_costo_gradas	2,0	Las gradas cuestan el doble
penalizacion_techado	1,5	Los tramos techados cuestan 1,5×
altura_piso_m	3,2	Altura de un piso
factor_congestion_demo	3,0	Multiplicador del escenario de congestión

> Los números del modelo y de ACO **no están escritos en el código**: salen de `config_modelo.json` y `config_aco.json`. Si preguntan «¿puedo cambiar X?», la respuesta es sí: cambia el JSON y vuelve a correr.

4. Dijkstra, explicado desde cero

Qué hace. Encuentra el camino de menor costo desde un origen. Mantiene una «cola de prioridad» de nodos por visitar, ordenada por el costo acumulado desde el origen. Cada vez saca el nodo con menor costo acumulado, lo da por definitivo (ya no se puede mejorar) y actualiza a sus vecinos si encuentra un camino más barato.

Versión multi-meta (la que usamos). Tenemos tres puntos de encuentro y no importa a cuál se llegue. Como Dijkstra saca los nodos en orden de costo creciente, basta detenerse cuando salga el primer punto de encuentro: es el más barato de alcanzar. Está en `src/dijkstra.py`.

Ejemplo con números

Aristas: A–P 3, P–G 8, G–S1 6, P–S2 20, S1–PE 30, S2–PE 12. Origen A, meta PE (nodos inventados para ilustrar).

Paso	Nodo que sale de la cola	Costo acumulado	Qué se actualiza
1	A	0	P = 3
2	P	3	G = 11, S2 = 23
3	G	11	S1 = 17
4	S1	17	PE = 47 (por S1)
5	S2	23	PE = 35 (mejora el 47)
6	PE	35	Meta alcanzada: ruta A→P→S2→PE, costo 35

Observa que Dijkstra **corrigió** su estimación de PE (47 → 35). Esa capacidad de corregir es lo que garantiza el óptimo.

Por qué es tan rápido aquí. Con 75 nodos casi no hay trabajo: 0,075 a 0,093 ms por ruta. Es determinista: misma entrada, misma salida. Por eso es nuestra **referencia**: el costo de cada ruta de ACO se compara con el de Dijkstra. Dijkstra nunca se usa dentro de ACO.

5. ACO (Optimización por Colonia de Hormigas), explicado desde cero

5.1 La idea biológica

Las hormigas reales buscan comida así: al principio caminan casi al azar y, por donde pasan, sueltan una sustancia llamada **feromona**. Otras hormigas tienden a seguir los caminos con más feromona. Como las hormigas que toman el camino corto vuelven antes, ese camino se refuerza más rápido; y la feromona se **evapora** con el tiempo, así que los caminos poco usados se olvidan. Resultado: el grupo «descubre» un buen camino sin que nadie lo planifique. ACO copia este mecanismo en un computador.

5.2 Los cuatro ingredientes

Ingrediente	Símbolo	Qué es en nuestro programa
Hormiga	–	Un recorrido aleatorio que parte del origen y termina en un punto de encuentro. En cada iteración salen 20.
Feromona	τ (tau)	Un número por arista (`tau` en `aco.py`). Alto = atractiva. Empieza en 1,0.
Visibilidad (heurística)	η (eta)	Qué tan buena parece una arista «a simple vista»: 1 ÷ costo (clásico).
Evaporación	ρ (rho)	Fracción de feromona que se pierde cada iteración (0,5 = pierde la mitad).

5.3 Cómo decide una hormiga a dónde ir

Parada en un nodo, la hormiga mira los vecinos a los que puede ir (los que no ha pisado ya: **lista tabú**) y elige uno con probabilidad proporcional a:

```
peso(j) = τ(i,j)^α × η(j)^β            probabilidad(j) = peso(j) ÷ suma de los pesos de todos los candidatos
```

- **α (alfa)**: cuánto le importa la feromona. α = 0 la ignora.
- **β (beta)**: cuánto le importa la visibilidad. β = 0 la ignora.

Ejemplo (números inventados). En el nodo P hay dos opciones: ir a G (arista de costo 8) o a S2 (costo 20). Al inicio τ = 1 en ambas, α = 1, β = 2 (clásico): pesos $1\times(1/8)^2 = 0{,}0156$ y $1\times(1/20)^2 = 0{,}0025$. Probabilidades: 86 % y 14 %. La hormiga prefiere la arista barata aunque lleve a un camino total más largo (ver ejemplo de la parte 4: por G se llega a 47, por S2 a 35). Esa **miopía** es el defecto del ACO clásico con β alto: premia lo barato *aquí y ahora*, no lo bueno en total.

5.4 Cómo se actualiza la feromona (al terminar cada iteración)

1. **Evaporación**: $\tau \leftarrow \max(\tau_{min}, (1-\rho)\cdot\tau)$ en todas las aristas.
2. **Depósito**: cada hormiga que llegó a un punto de encuentro suma Q ÷ (costo de su camino) a cada arista que usó. Camino más corto → depósito mayor.
3. **Refuerzo élite**: la mejor hormiga de la iteración deposita una segunda vez.

4. **Límites (variante MAX-MIN)**: τ nunca baja de τ_min = 0,01 ni sube de τ_max = 10. Evita que una arista quede en probabilidad cero y que una sola ruta lo domine todo.

5.5 Cuándo termina

Después de 100 iteraciones, o antes si pasan 25 iteraciones seguidas sin mejorar el mejor costo (paciencia). Por eso una corrida que encuentra el óptimo en la iteración 1 igual ejecuta unas 26 iteraciones. La respuesta final es el mejor camino visto en toda la corrida.

5.6 Variable por variable (ACO)

Variable	Valor	Para qué sirve	Qué pasa si la subes
n_hormigas	20	Recorridos por iteración	Más éxito y más tiempo (clásico, 10 orígenes: 5→22 % y 2,3 ms; 50→51 % y 17 ms)
n_iteraciones	100	Máximo de rondas	Más tiempo si no converge
alfa (α)	1,0	Peso de la feromona	Sin efecto medible con Q = 1
beta (β)	2,0 clásico / 1,0 guiado	Peso de la visibilidad	En el clásico empeora el éxito (β=0: 99 %, β=5: 14 %)
rho (ρ)	0,5	Evaporación	Sin efecto medible con Q = 1
Q	1,0	Escala del depósito (Q ÷ costo)	Q grande refuerza rutas tempranas, aunque sean malas: la brecha sube (1,92 %→16,34 % con β=0)
tau_inicial / tau_min / tau_max	1,0 / 0,01 / 10,0	Feromona inicial y límites	–
paciencia_estancamiento	25	Iteraciones sin mejora para detenerse	Más tiempo
max_pasos_factor	4	Límite de pasos por hormiga = 4 × nodos	Evita que el retroceso dure para siempre

¿Por qué α y ρ no cambiaron nada? Con Q = 1 y rutas de costo ≈ 200, cada depósito es ≈ 1/200 = 0,005, muy pequeño frente a la feromona inicial (1,0) y al mínimo (0,01). La feromona casi no distingue caminos (en la figura 2 se ve: ya en la iteración 21 todas valen 0,010). Lo comprobamos con el diagnóstico de Q: al subir Q la feromona sí influye, pero refuerza malas rutas tempranas y la brecha empeora. Es una explicación respaldada por ese experimento, no una demostración formal.

6. ACO clásico frente a ACO guiado

6.1 ACO clásico (el de libro)

Es ACO sin ayudas: la hormiga elige con τ y η = 1 ÷ costo; si se queda sin vecinos (por ejemplo, entra a un aula, que es un callejón), **muere** y no deja feromona. Si ninguna de las 20 hormigas llega a un punto de encuentro en ninguna iteración, la corrida falla. Parámetros: α=1, β=2, ρ=0,5, 20 hormigas, 100 iteraciones. Lo declaramos antes de medir (β=2 es el valor estándar de la literatura), no lo ajustamos para que salga bien o mal.

6.2 ACO guiado: las tres mejoras (y por qué se llama «guiado»)

Mejora	Qué hace	Por qué ayuda
1. Poda de callejones	Antes de empezar, quita del grafo los nodos de grado 1 (hojas) que no sean el origen ni un punto de encuentro, repitiendo hasta que no queden. Se hace sobre una vista; el grafo original no se modifica.	Un nodo hoja nunca está en el camino óptimo entre otros dos nodos, así que podarlo nunca desconecta el origen de una meta. Elimina las aulas ajenas como trampas.
2. Retroceso	Si la hormiga queda sin vecinos, retrocede un paso y marca ese nodo como muerto solo para ella; solo falla si vuelve al origen sin opciones o supera el límite de pasos (4 × nodos).	Una hormiga atrapada ya no se pierde; busca otra salida.
3. Heurística guiada (tipo A*)	η = 1 ÷ (costo de la arista + h(j)), donde h(j) es la distancia en línea recta de j al punto de encuentro más cercano. β = 1.	La hormiga prefiere vecinos que la acercan geométricamente a la meta, no solo la arista local más barata. Es la misma idea de A*.

«Guiado» viene de esa tercera mejora: el algoritmo recibe una pista de hacia dónde queda la meta. Las tres juntas también le quitan las trampas, por eso lo llamamos así. **No llama a Dijkstra**: usa solo la geometría del plano.

6.3 Comparación lado a lado

	Dijkstra	ACO clásico	ACO guiado
Tipo	Exacto, determinista	Metaheurística, estocástico	Metaheurística con conocimiento del problema
Garantiza óptimo	Sí	No	No (pero lo logró 7560/7560)
Éxito (encuentra ruta)	100 %	11 a 22 %	100 %
Brecha media vs. óptimo	0 %	5 a 17 % (solo exitosas)	0,00 %
Tiempo por ruta	0,075 a 0,093 ms	5,0 a 5,9 ms	13,4 a 29,6 ms
Iteración de convergencia	–	8,7 a 12,0	1,0 a 1,7
Usa feromona	No	Sí	Sí

¿Cuál es mejor? Para este problema, **Dijkstra**: óptimo y ~264 veces más rápido. Entre las dos variantes de ACO, **el guiado**, sin duda. El clásico es más rápido por corrida solo porque sus hormigas mueren pronto, no porque trabaje mejor.

> **Aclaración importante.** El guiado llega a cero brecha casi siempre en la iteración 1. Eso indica que casi no necesita las feromonas: poda + retroceso + heurística ya llevan a la solución. Por eso la conclusión de la lámina 12 dice que la poda y el retroceso explican el óptimo «más que las feromonas».

7. El agente de evacuación

Está en `src/agente.py` (clase `AgenteEvacuacion`) y sigue el ciclo percibir → decidir → actuar:

- `percibir(EstadoPercibido)`: recibe las aristas bloqueadas y un mapa de congestión.
- `decidir()`: aplica la congestión al grafo (multiplica la distancia de esas aristas), corre ACO (variante a elección; puede reutilizar la feromona anterior: *warm start*) y, si se pide, corre también Dijkstra solo para comparar.
- `actuar()`: convierte el camino en instrucciones («Sale del H203, toma gradas, sale por SALIDA-B…») y calcula el tiempo estimado; si no hay ruta, lo avisa con un error explícito.

PEAS	En este trabajo
Rendimiento	Llegar a un punto de encuentro, minimizar el costo, evitar bloqueos, avisar si no hay ruta, tiempos de primera y última persona
Entorno	El complejo G/H como grafo de 75 nodos y 79 aristas, con bloqueos o congestión
Actuadores	`decidir()` (calcula) y `actuar()` (instrucciones y tiempo)
Sensores	`percibir()`: aristas bloqueadas y congestión

Propiedad del entorno	Clasificación	Por qué
Observable	Parcialmente	Solo conoce lo que le reportan
Determinista	Estocástico	ACO usa azar; el mundo real es impredecible
Episódico	Secuencial	Cada paso depende de los anteriores
Estático	Dinámico	Pueden aparecer bloqueos entre dos decisiones
Discreto	Discreto	Nodos y aristas finitos
Agentes	Multiagente cooperativo	En la realidad hay muchas personas; aquí se modela una por origen
Conocido	Parcialmente	Reglas conocidas; PEs y parte del grafo son supuestos

Tipo de agente: basado en **metas** (llegar a un PE) y en **utilidad** (entre varias rutas, la de menor costo). **Qué NO es:** no simula a 1020 personas interactuando, no modela colas ni capacidad de gradas, no aprende entre simulacros, y Dijkstra se usa únicamente como referencia de comparación, nunca para decidir.

8. Cómo funciona el programa de Python

8.1 Mapa de archivos

Archivo	Qué hace
`data/campus_graph.json`	El grafo: 75 nodos y 79 aristas
`data/config_modelo.json`	Supuestos del modelo (velocidad, reacción, población, costos…)
`data/config_aco.json`	Parámetros de ACO y plan de experimentos
`src/grafo.py`	Carga el grafo; método `vecinos()` que ignora aristas bloqueadas; `aplicar_congestion()`
`src/costos.py`	Función de costo y de tiempo (única para todos)
`src/dijkstra.py`	Dijkstra multi-meta con cola de prioridad (`heapq`)
`src/aco.py`	ACO: `aco_iterativo()` (generador, una iteración a la vez), `ejecutar_aco()`, `podar_callejones()`, `calcular_heuristica()`, `_construir_camino_hormiga()`
`src/agente.py`	Agente: percibir, decidir, actuar
`src/escenarios.py`	Define los 6 escenarios (qué aristas se bloquean)
`src/benchmark.py`	Corre muchas veces los algoritmos y calcula brecha, éxito y tiempo
`src/evacuacion.py` y `src/metricas.py`	Tiempo de llegada de la primera y la última persona; curva acumulada
`src/plano.py`	Dibuja rutas sobre el plano calibrado y el esquema por niveles
`scripts/correr_experimentos.py`	Corre todos los experimentos oficiales (~8 min) y escribe `results/*.csv`
`scripts/generar_figuras.py`	Lee los CSV y genera las figuras y los GIF
`app/app.py`	App web en Streamlit (4 pestañas)
`tests/`	86 pruebas automáticas (`python -m pytest`)

8.2 Qué pasa por dentro, paso a paso

1. Se cargan el grafo y los dos JSON de configuración.
2. El escenario elegido bloquea aristas (`aristas_bloqueadas`) o infla su distancia (congestión).
3. **Dijkstra**: usa la cola de prioridad y devuelve camino, costo y punto de encuentro.
4. **ACO** (`aco_iterativo`): (a) en el clásico no se poda; en el guiado se poda la vista del grafo; (b) se calcula h(j) para el guiado; (c) se crea la feromona τ = 1 en cada arista; (d) en cada iteración salen 20 hormigas; cada una construye su camino con `_construir_camino_hormiga` (pila de nodos = camino actual, `activos` = lista tabú, `muertos` = nodos descartados al retroceder); (e) se evapora, se deposita y se limita τ; (f) se guarda el mejor camino global; (g) se detiene por iteraciones o por paciencia.
5. Se compara: **brecha % = (costo ACO − costo Dijkstra) ÷ costo Dijkstra × 100**.

8.3 Variables importantes dentro del código

Nombre	Significado
`tau`	Diccionario arista → feromona
`heuristica`	Diccionario nodo → h(j) (ceros en el clásico)
`pila`	Camino que lleva la hormiga (último = nodo actual)
`activos`	Nodos del camino actual (no se repiten: lista tabú)
`muertos`	Nodos que esa hormiga descartó al retroceder
`candidatos`	Vecinos permitidos en este paso
`variante`	"clasico" o "guiado"
`semilla`	Semilla del generador aleatorio
`aristas_bloqueadas`	Conjunto de ids de aristas cerradas
`iteracion_mejor_solucion`	Iteración en que se halló el mejor camino (la «convergencia»)

8.4 Cómo ejecutarlo

```
pip install -r requirements.txt            # una sola vez
python -m pytest                           # 86 pruebas
streamlit run app/app.py                   # abre la app en el navegador
python scripts/correr_experimentos.py      # recalcula todo (~8 min) → results/
python scripts/generar_figuras.py          # regenera figuras y GIF
```

8.5 La app (pestañas) y una demo de 2 minutos para clase

Pestaña	Qué muestra
Simulación en vivo	Iteración por iteración: mejor costo de ACO, óptimo de Dijkstra, brecha, hormigas con ruta; el plano con las rutas y el mapa de feromona por nivel
Comparación	Una tabla Dijkstra vs. la variante elegida (costo, tiempo de cómputo, iteración, salida, punto de encuentro, tiempo de recorrido) y la brecha
Benchmark	Resultados guardados (tablas, figuras, animaciones) y re-ejecución pequeña que no pisa los CSV oficiales
Evacuación completa	Curva de llegadas de las 1020 personas por algoritmo y escenario; reparto por salida y por punto de encuentro

> **Demo sugerida.** (1) Barra lateral: origen **H403**, escenario **Normal**, variante **clásico**. En «Simulación en vivo» pulsa ejecutar: se verá que muchas corridas no llegan o llegan tarde. (2) Cambia a **guiado** y repite: la línea de mejor costo cae al óptimo de Dijkstra en 1 a 4 iteraciones. (3) Cambia el escenario a **Salida B bloqueada**: la ruta pasa por la Salida A y el costo sube. (4) En «Evacuación completa», muestra la curva y las métricas de primera y última persona. Mueve β en el clásico para mostrar que con β=0 mejora.

9. Qué medimos (definiciones)

Métrica	Definición	Cómo leerla
Tasa de éxito	% de corridas de ACO que encontraron alguna ruta a un punto de encuentro	100 % = siempre llega. Una ruta peor es éxito; no llegar es fallo.
Costo	Suma de costos de aristas (metros equivalentes)	Menor es mejor. El mínimo posible lo da Dijkstra.
Brecha (%)	(costo ACO − costo Dijkstra) ÷ costo Dijkstra × 100	0 % = óptimo. Se calcula solo en corridas exitosas, por eso puede verse bajo aunque el éxito sea malo.
% de corridas óptimas	Corridas con brecha 0	Guiado: 100 %. Clásico: 8 a 15 %.
Iteración de convergencia	Iteración en que ACO halló su mejor ruta final	Menor = más rápido en rondas.
Tiempo de cómputo (ms)	Tiempo del algoritmo (no de la evacuación)	Dijkstra: mediana de 30 repeticiones; ACO: por corrida.
Primera / última persona	Segundos desde la alerta hasta que llega la primera / la última de las 1020 personas	Son las métricas de la guía de simulacros.
Prueba de cordura	Verifica que ninguna ruta de ACO cueste menos que la de Dijkstra	20.077 corridas exitosas, 0 violaciones: la comparación es coherente.

Diseño de los experimentos: 6 escenarios × 42 orígenes × 30 corridas × 2 variantes = **15.120 corridas** (7560 por variante). Aparte: adaptación (arranque en frío vs. warm start), sensibilidad de α, β, ρ y número de hormigas (solo ACO clásico, 10 orígenes), diagnóstico de Q y evacuación completa.

10. Las gráficas, una por una

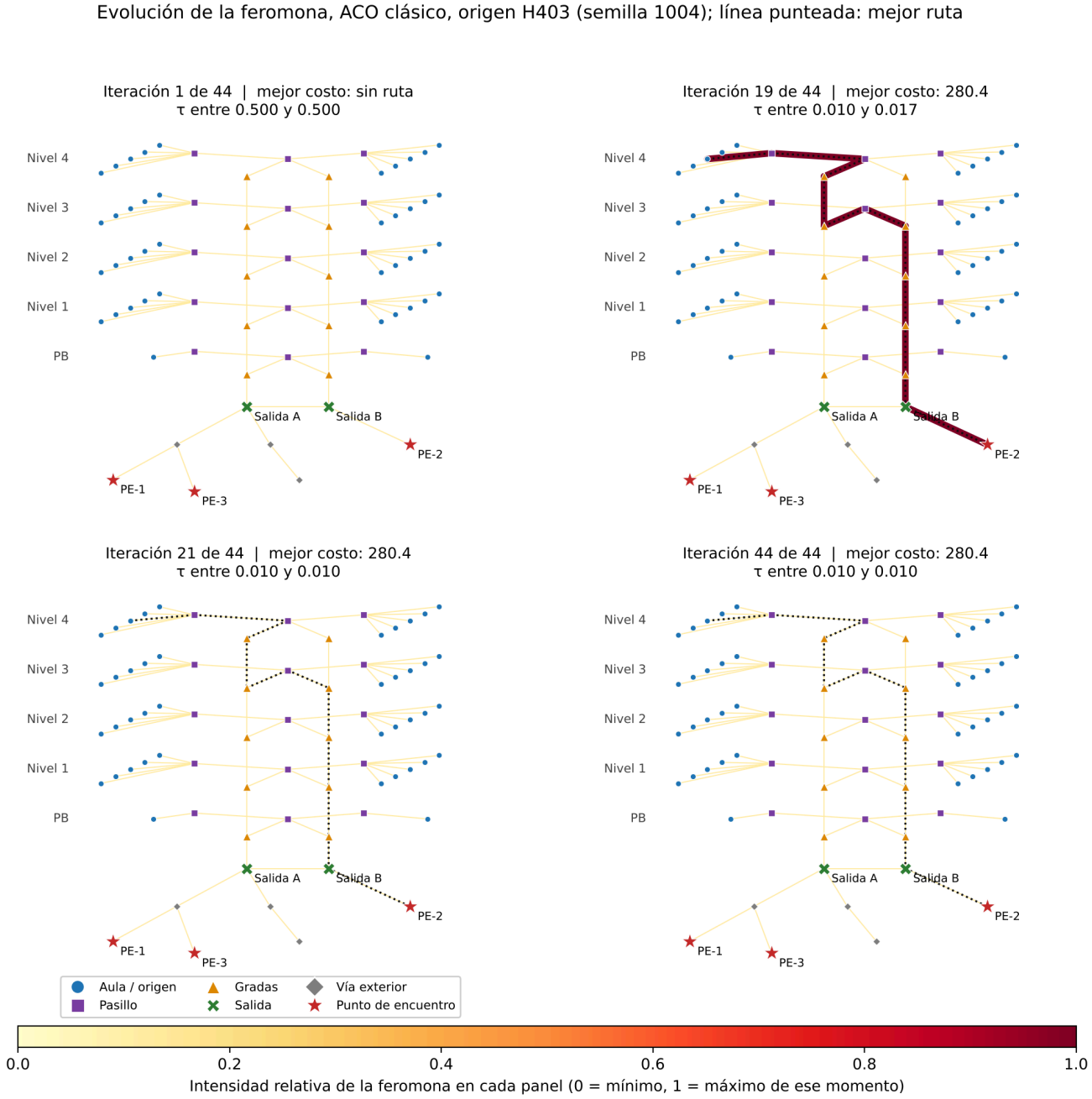
Cada figura está en `results/figuras/` y se genera con `scripts/generar_figuras.py` a partir de los CSV.

Figura 1: rutas sobre el plano calibrado (láminas 1 y 4)



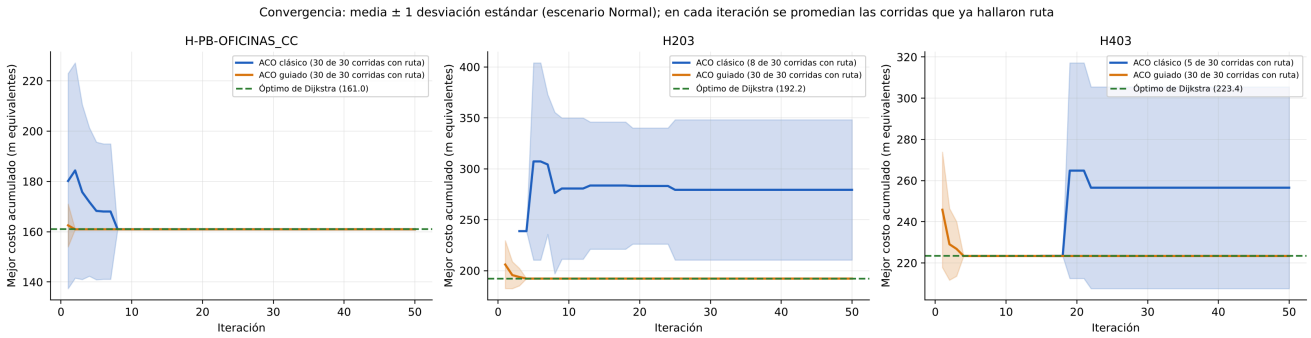
Qué muestra: la ruta de cada método desde tres orígenes (planta baja, H203 y H403), sobre la foto del plano ya calibrada en metros. **Cómo leerla:** línea verde gruesa = Dijkstra (óptimo); azul rayada = ACO clásico; naranja punteada = ACO guiado. Círculo = origen, cuadro = pasillo, triángulo = gradas, X verde = salida. El cuadro de abajo dice a qué punto de encuentro llegó cada método. **Qué concluir:** guiado y Dijkstra se superponen; el clásico desde H203 se desvía hasta PE-1.

Figura 2: mapa de calor de feromonas (GIF y panel)



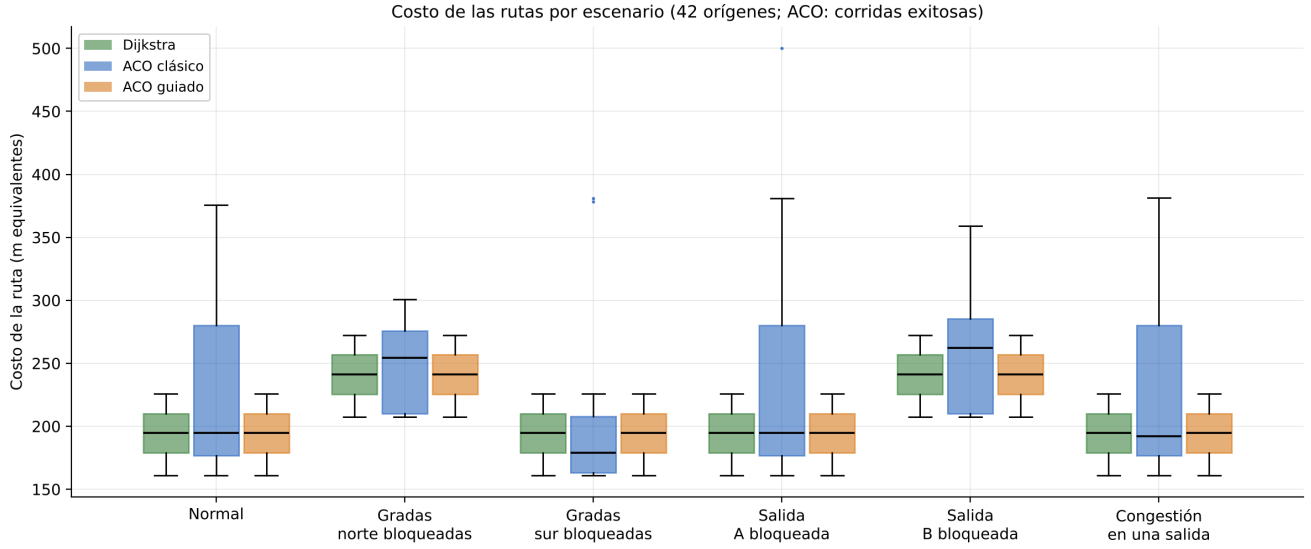
Qué muestra: cómo evoluciona la feromona de ACO clásico desde H403, en cuatro momentos, sobre un esquema por niveles (nivel 4 arriba, planta baja abajo). **Cómo leerla:** color claro = poca feromona, rojo oscuro = mucha, normalizado dentro de cada panel; la línea punteada es la mejor ruta. En la iteración 1 todas las aristas valen lo mismo (0,5 después de evaporar); en la 19 aparece la ruta marcada; en la 21 y la 44 la feromona ya está en su mínimo (0,010) en todas partes. **Qué concluir:** con Q = 1 la feromona se desvanece y casi no guía: apoya el hallazgo de que α y ρ no influyen. Hay una versión animada en `results/animaciones/aco_feromonas.gif`.

Figura 3: convergencia (lámina 8)



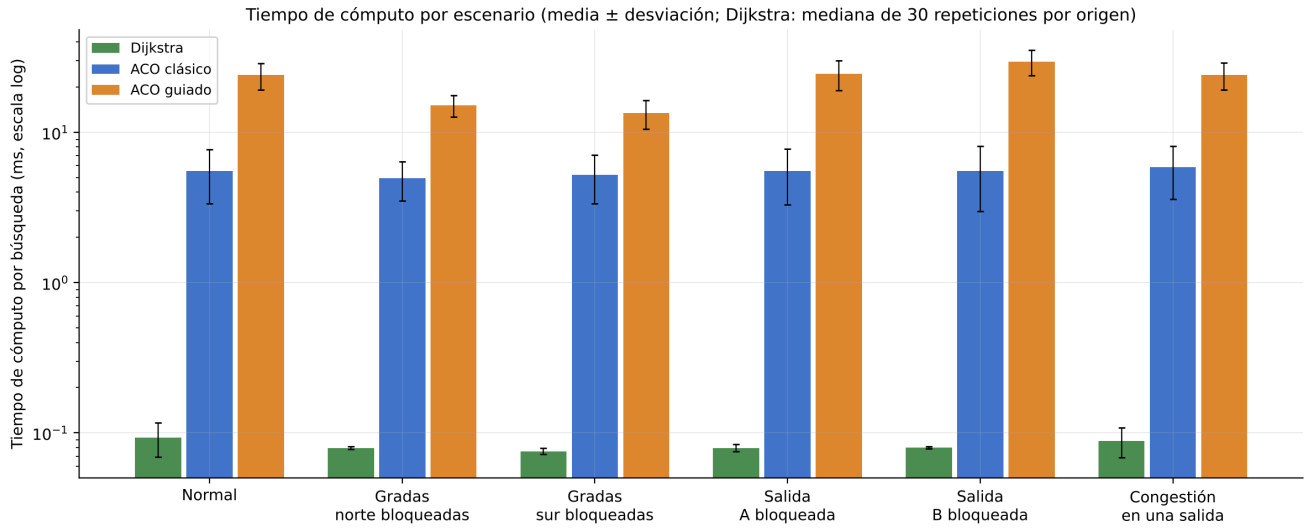
Qué muestra: cómo baja el mejor costo con las iteraciones, para tres orígenes. Eje X: iteración. Eje Y: mejor costo acumulado. Línea punteada verde: el óptimo de Dijkstra. Bandas: ±1 desviación estándar. En la leyenda aparece «n de 30 corridas con ruta». **Qué concluir:** el guiado se pega al óptimo en 1 a 4 iteraciones; el clásico, desde H203 y H403, casi nunca encuentra ruta (8 y 5 de 30) y cuando la halla queda lejos del óptimo.

Figura 4a: costos por escenario (diagrama de cajas)



Cómo leerla: cada caja contiene el 50 % central de los costos; la línea negra es la mediana; los bigotes, el rango; los puntos sueltos, casos extremos. Verde = Dijkstra, azul = clásico, naranja = guiado. **Qué concluir:** guiado y Dijkstra son cajas idénticas; el clásico es mucho más ancho (rutas muy variables y a menudo caras). Los costos suben a ~245 con la Salida B o las gradas norte bloqueadas porque todos deben ir por la Salida A.

Figura 4b: tiempo de cómputo (escala logarítmica)

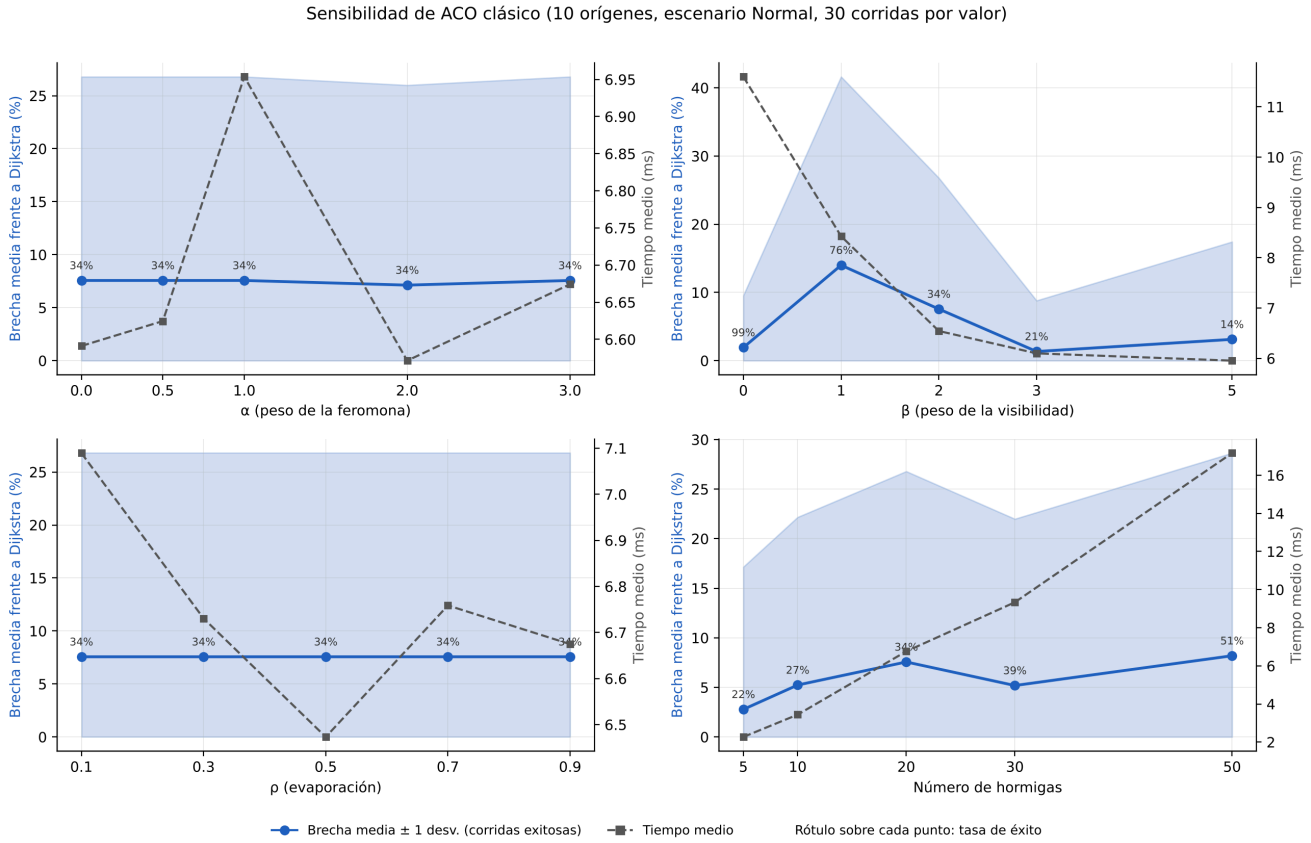


Cómo leerla: el eje vertical es logarítmico: cada marca es 10 veces mayor que la anterior. Dijkstra ≈ 0,08 ms; clásico ≈ 5 a 6 ms; guiado ≈ 13 a 30 ms. **Qué concluir:** Dijkstra es ~264 veces más rápido que el guiado. El clásico es más rápido que el guiado solo porque sus hormigas se mueren pronto.

Figura 4c: éxito y calidad (lámina 7)

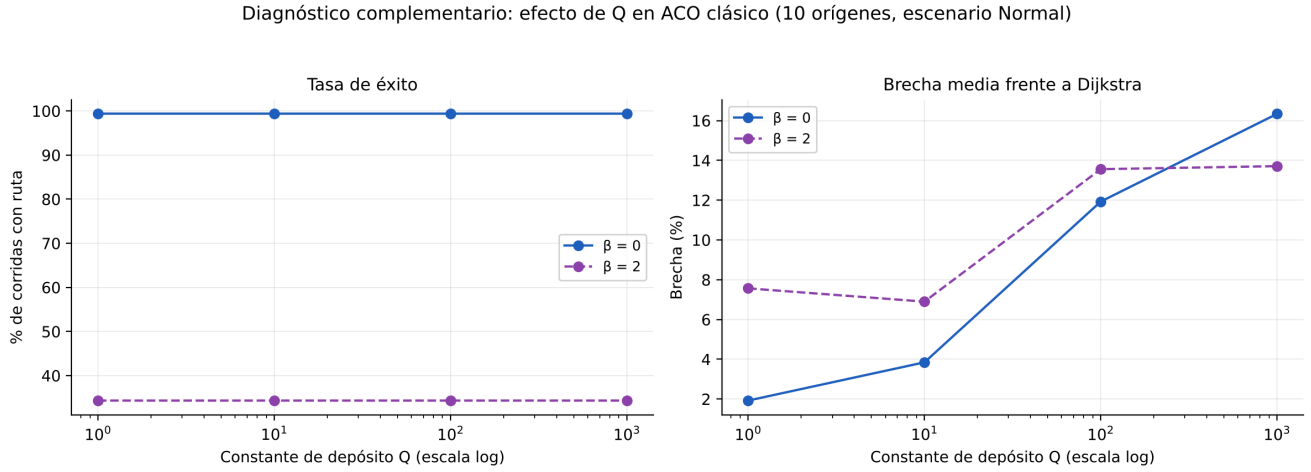
Izquierda: tasa de éxito por escenario (guiado 100 %; clásico 11 a 22 %). **Derecha:** brecha frente a Dijkstra con su desviación (guiado 0; clásico 5 a 17 % de media, hasta 157 % en el peor caso). Recuerda que la brecha solo cuenta corridas exitosas.

Figura 5: sensibilidad de parámetros (lámina 9)



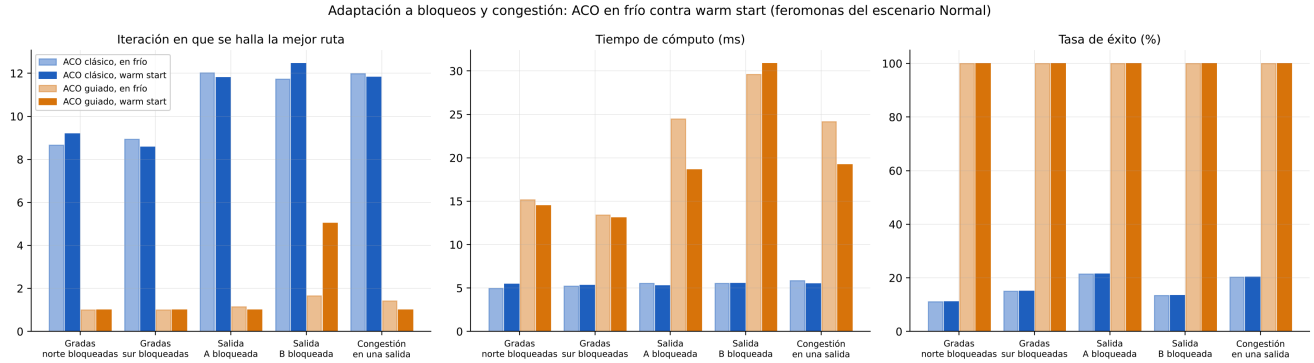
Qué muestra: cuatro paneles, uno por parámetro (α, β, ρ, número de hormigas), para el ACO clásico en 10 orígenes. **Cómo leerla:** línea azul = brecha media (banda = ±1 desviación); línea gris punteada = tiempo medio (eje derecho); el rótulo sobre cada punto es la tasa de éxito. **Qué concluir:** β es el único con efecto grande (β=0 → 99 %; β=5 → 14 %); α y ρ son líneas planas (34 %); más hormigas aumenta el éxito pero también el tiempo. Cuidado: una brecha pequeña con β=3 (1,3 %) viene con solo 21 % de éxito, o sea que se mide en pocas corridas.

Figura 5b: diagnóstico de Q (complementaria)



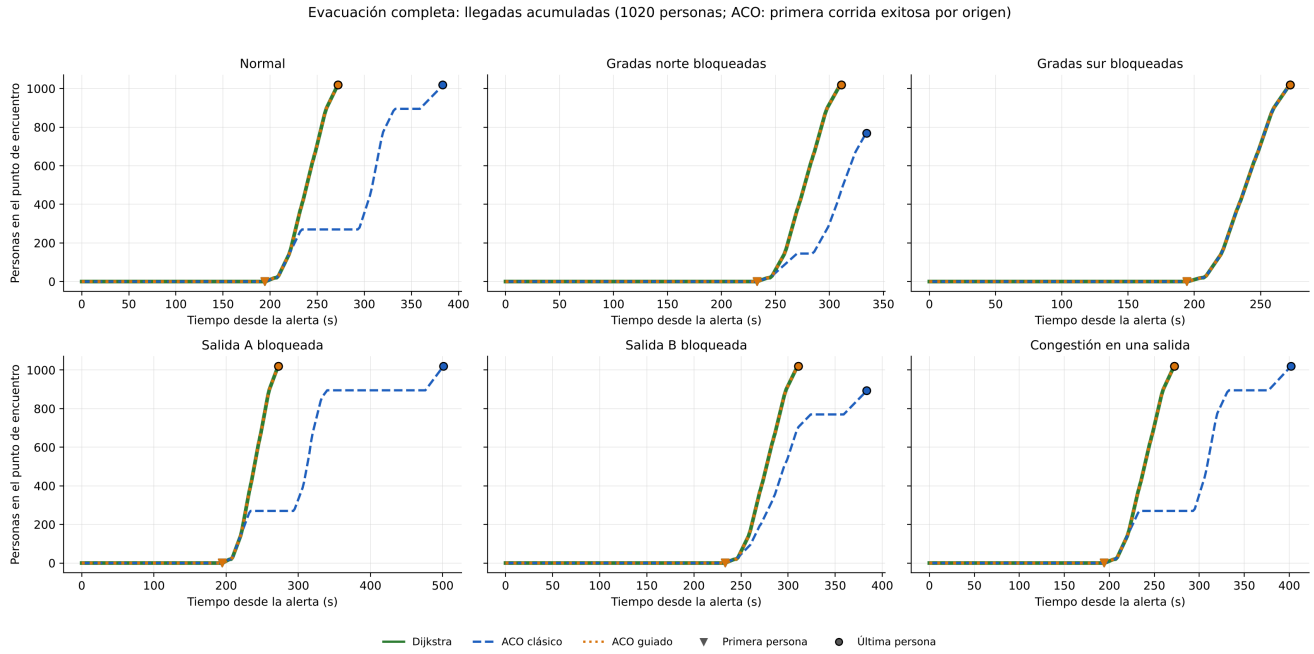
Qué muestra: qué pasa si se cambia la constante de depósito Q (1, 10, 100, 1000), con β = 0 y β = 2. **Qué concluir:** el éxito no cambia (99,3 % y 34,3 %), pero la brecha crece cuando Q es grande: un depósito fuerte «fija» la colonia en las primeras rutas, aunque sean malas. Se hizo para explicar por qué α y ρ no tenían efecto; no estaba en el plan original.

Figura 6: frío contra warm start (lámina 10)



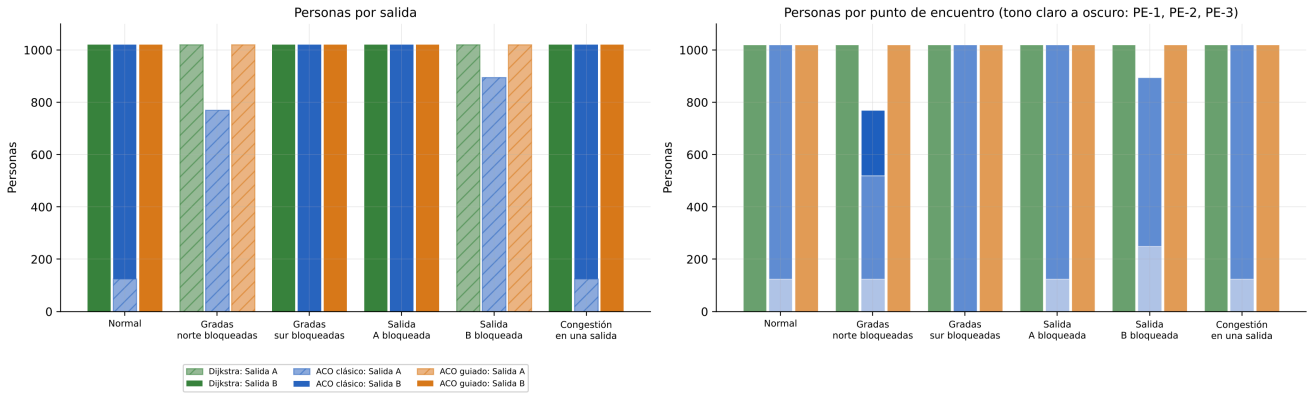
Qué muestra: si reutilizar la feromona de la corrida «Normal» ayuda cuando aparece un bloqueo. Tres paneles: iteración en que se halla la mejor ruta, tiempo y tasa de éxito; barras claras = arranque en frío, oscuras = warm start. **Qué concluir:** en el guiado ahorra ~20 % de tiempo cuando la ruta principal está libre (Salida A bloqueada, congestión), pero no ayuda cuando B está cerrada (1,67 → 5,04 iteraciones), porque la feromona apunta a la ruta cerrada. En el clásico no hay ventaja sistemática.

Figura 7: llegadas acumuladas de las 1020 personas (lámina 11)



Cómo leerla: eje X: segundos desde la alerta; eje Y: personas que ya llegaron al punto de encuentro. Triángulo = primera persona, círculo = última. **Qué concluir:** normal: 194 s la primera y 272 s la última; con Salida B o gradas norte bloqueadas: 233 s y 311 s (+39 s). La curva del clásico se queda corta (770 u 895 personas) o llega tarde, porque algunos orígenes no encuentran ruta.

Figura 7b: personas por salida y punto de encuentro



Qué muestra: cuántas de las 1020 personas usan cada salida (izquierda) y cada punto de encuentro (derecha). Con Dijkstra y guiado, **todos** van por la Salida B y llegan a PE-2 en el escenario normal; con B o gradas norte bloqueadas, todos por la Salida A. El clásico reparte a algunos por PE-1 y PE-3, incluso por rutas peores. **Ojo:** que todos usen la misma salida es consecuencia del modelo (PE hipotéticos, sin colas ni capacidad); en la realidad se saturaría.

Animaciones: `aco_feromonas.gif` (la feromona evolucionando) y `evacuacion.gif` (llegada de las personas). Las usamos para mostrar el proceso en vivo.

11. Qué significan los resultados

Hallazgo	Dato	Interpretación
ACO guiado = óptimo	100 % éxito, 0 % brecha en 7560 corridas	Con poda, retroceso y heurística, ACO encuentra siempre la ruta óptima en este grafo.
ACO clásico falla mucho	11 a 22 % de éxito; brecha máx. 157 %	Las hormigas entran a callejones (aulas) y mueren; sin retroceso no hay forma de salir.
Dijkstra gana en velocidad	0,08 ms vs 13 a 30 ms (~264×)	En un grafo pequeño y fijo, el algoritmo exacto es la mejor opción.
El guiado converge casi al instante	Iteración 1,0 a 1,7	La feromona aporta poco; lo que funciona es el conocimiento embebido.
β domina en el clásico	β=0: 99 %; β=2: 34 %; β=5: 14 %	Una visibilidad fuerte atrae a las aristas cortas, que suelen ser callejones.
α y ρ sin efecto (Q=1)	34,3 % siempre	El depósito (~0,005) es insignificante frente al mínimo de feromona.
Q grande empeora	Brecha 1,92 % → 16,34 %	Refuerza pronto rutas malas (convergencia prematura).
Bloquear B o las gradas norte	Costo medio 199,3 → 245,9 (+23 %); +39 s	Toda la gente debe usar la Salida A, que es más lejana.
Warm start	−20 % de tiempo con ruta libre; peor si B cerrada	Reutilizar el pasado ayuda solo si sigue siendo válido.
Tiempos de evacuación	Normal: 194 s y 272 s. Bloqueo: 233 s y 311 s	Estimación con un modelo simple, sin colas.

12. Límites del trabajo (hay que decirlos)

- **Puntos de encuentro hipotéticos.** Son tres supuestos fuera de los bloques; con ellos, todos los orígenes terminan en PE-2 vía Salida B. No es un plan oficial.
- **Sin colas ni capacidad.** No modelamos aglomeración en gradas o salidas, así que los tiempos son optimistas. La única «congestión» es el factor ×3 de un escenario.
- **Una persona por origen.** No hay 1020 agentes interactuando.
- **Sin aprendizaje entre simulacros.** El agente solo reutiliza feromona dentro de una sesión.
- **Grafo y supuestos.** Datos a partir de un plano calibrado (error 2,19 %), con costos supuestos (gradas ×2, techado ×1,5, 25 personas por aula).
- **Python, un hilo.** Los tiempos de ms dependen de la máquina; sirven para comparar entre métodos, no como valores absolutos.
- **La comparación no es «ACO malo, Dijkstra bueno».** Es: un buen conocimiento del problema pesa más que la feromona, y en un grafo chico un algoritmo exacto basta.

13. Preguntas probables y cómo contestarlas

Pregunta	Respuesta corta
¿Qué es ACO en una frase?	Una metaheurística donde «hormigas» virtuales construyen caminos al azar y dejan feromona; los caminos cortos reciben más feromona y atraen a más hormigas, mientras la evaporación olvida los malos.
¿Qué es Dijkstra?	Un algoritmo exacto que expande siempre el nodo de menor costo acumulado y garantiza la ruta óptima; aquí se detiene en el primer punto de encuentro.
¿Qué diferencia hay entre clásico y guiado?	El guiado poda callejones, retrocede cuando se atasca y suma una heurística de distancia a la meta; el clásico no hace nada de eso.
¿Por qué «guiado»?	Porque su visibilidad lleva una pista geométrica de dónde está la meta (idea de A*), además de la poda y el retroceso.
¿Cuál es mejor?	Dijkstra para este problema (óptimo y 264× más rápido). Entre los dos ACO, el guiado: 100 % éxito y 0 % brecha frente a 11 a 22 % de éxito.
Si Dijkstra es mejor, ¿para qué ACO?	Para estudiar la metaheurística y ver sus condiciones de éxito. ACO resulta útil en problemas más grandes, dinámicos o sin algoritmo exacto eficiente. Aquí no es necesario.
¿El guiado hace trampa usando Dijkstra?	No. No llama a Dijkstra. Usa la distancia euclidiana al punto de encuentro (como A*) y poda hojas del grafo.
¿Por qué el clásico usa β=2?	Es el valor estándar de la literatura (Dorigo y Stützle, 2004), declarado antes de medir. Con β=0 el clásico sería mucho mejor (99 %), pero eso lo mostramos como análisis de sensibilidad, no como configuración base.
¿Por qué α y ρ no influyen?	Con Q=1 el depósito (≈0,005) es mínimo frente a la feromona y llega al piso (0,01); la feromona queda casi uniforme. Con Q grande sí influye, pero empeora.
¿Qué es la brecha?	Cuánto más cara es la ruta de ACO respecto a la óptima de Dijkstra, en %. Solo se calcula en corridas que llegaron.
¿Qué es warm start?	Empezar ACO con la feromona de una ejecución anterior en vez de uniforme. Ayuda si la ruta sigue válida; perjudica si apunta a una ruta bloqueada.
¿Cómo validaron que ACO no «inventa» rutas mejores?	Con una prueba de cordura: en 20.077 corridas exitosas ninguna fue más barata que Dijkstra.
¿Cómo se calibró el plano?	Con la distancia real entre las dos salidas (61,7 m) se obtuvo 6,948 px/m; la barra de escala (71 px = 10 m) la verificó con 2,19 % de error.
¿Por qué todos van a la Salida B?	Por la ubicación hipotética de los PE y porque no modelamos colas; es un límite declarado, no una conclusión sobre el edificio real.
¿Cómo se calcula el tiempo de evacuación?	60 s de reacción + costo de la ruta ÷ 1,2 m/s para la primera persona de un origen; cada persona siguiente llega 1 s después.
¿Es un agente inteligente?	Es un agente basado en metas y utilidad: percibe bloqueos, planifica una ruta y comunica instrucciones. No aprende ni negocia con otros.
¿Qué falta?	Validar PEs reales, modelar colas y aforo de gradas, simular personas interactuando y hacer pruebas en campo (simulacros).

14. Glosario

Término	Significado
Arista	Conexión entre dos nodos con un costo
Brecha	% extra de costo respecto al óptimo
Callejón / hoja	Nodo con una sola conexión (ej. un aula)
Convergencia	Momento en que ACO halla su mejor ruta
Costo (m equivalentes)	Distancia con penalización por gradas y techado
Evaporación (ρ)	Pérdida de feromona por iteración
Feromona (τ)	Valor por arista que acumula la «memoria» de la colonia
Heurística (h)	Estimación de lo que falta para llegar a la meta
Lista tabú	Nodos ya visitados que la hormiga no repite
MAX-MIN	Variante de ACO con feromona acotada (0,01 a 10)
Metaheurística	Estrategia de búsqueda general, sin garantía de óptimo
Origen / punto de encuentro	Dónde parte una persona / dónde debe llegar
PEAS	Rendimiento, Entorno, Actuadores, Sensores
Poda	Eliminar nodos hoja del grafo antes de buscar
Retroceso	Volver un paso atrás al quedar atascado
Semilla	Valor que fija el azar para repetir resultados
Visibilidad (η)	Atractivo «a simple vista» de una arista
Warm start	Arrancar con la feromona anterior

Datos tomados de `results/` (benchmark de 15.120 corridas, sensibilidad, adaptación, evacuación) y del código en `src/`. Los ejemplos numéricos de Dijkstra y de probabilidades de ACO son ilustrativos (nodos inventados).